

The Needle and the HAYSTACK

Part—2

In the previous article, we demystified some search-related jargon, and learned how the humble Grep can be used to simulate a Boolean-model search engine. In this concluding article on the subject, let us explore two more search models that sort search results by relevance and allow for some custom tuning to suit the corpus. We also knock on the doors of content management without actually entering, just to show how the business of search becomes more fruitful when content is made to go that extra mile.

Let's begin by understanding the vector space model. In this model [1], each document is treated as a projection in vector space. A brief explanation is in order. An arbitrary point in a three-dimensional space can be represented by three coordinates: x, y, and z. Here, our vector space has three dimensions, and any point is the result of its three component vectors (coordinates) projected along their three respective axes.

Now imagine a vector space with more than three dimensions—in fact, as many dimensions as there are unique terms in the collection that you want to search. Each unique term in the collection represents an axis or a dimension. Now, every document can be 'visualised' as the result of all component vectors pointing along axes corresponding to unique terms in the document. This model treats every document as a bag of words, with no regard to its semantics or word order. For instance, the two documents *Cats love mice* and *Mice love cats* will have the same document vector.

The magnitude of a particular component vector (remember, each component vector represents a unique term now) for a document along that term's axis can now be made to depend on a 'weight' for that term, in that document. Term weight in a document is nothing but a measure of how important a particular term is to a document. Term weighting helps differentiate documents in vector space, making it easier to rank them by so-called relevance. Term weighting can get quite sophisticated, and we will dwell on it briefly later. The simplest term-weighting scheme is term frequency. An example should illustrate the concept. Let us modify our documents a little; for instance, they could be:

Cats love mice and I love cats.

...and,

Mice love cats.

Though both the documents have similar vectors in term space (ignoring 'I', and 'and'), the first document has higher weights for *cats* and *love*, based on in-document term frequency, making it more relevant in a search for, say, *cats*. In other words, a term with a high weight in a document makes that document more relevant in a search with that term as a search key-word.

Using the model

We now have a collection with a set of vectors, one to each document. Vector algebra gives us the means to compute the 'similarity' of two vectors, with the help of the dot product. A standard text on vector algebra will help you with more details on the vector dot product. But if you accept the maths on faith, then the following illustration should do for the moment. A dot product of two vectors $A = [a1, a2, a3, \dots, an]$ and $B = [b1, b2, b3, \dots, bn]$ is a scalar, and is given by the following equation:

$$A \cdot B = |A| |B| \cos \theta = a1b1 + a2b2 + a3b3 + \dots + anbn$$

...where θ is the angle between the vectors. The smaller the angle, the closer the vectors are to each other. And so, we have:

$$\cos \theta = (A \cdot B) / (|A| |B|)$$

Here, $|A|$ is the magnitude of A given by $\sqrt{a1^2 + a2^2 + \dots + an^2}$ with the magnitude of $|B|$ also being calculated in a corresponding manner.

For the purpose of our search, we need to ascertain how close our search term vector is to various documents in the collection, using the cosine similarity formula above. The cosine similarity is a normalised measure, making it independent of the size of the vectors—and consequently, the document sizes. The document (vector) closest to our search term vector ranks the highest in our search results.

Now that we have spoken enough about this model, let us take it for a test drive. Remember those four documents we